PEFT（参数高效微调）前置知识

> 这份文档为零基础读者准备。如果你已经熟悉微调、LoRA、QLoRA 等概念，可以直接翻阅主报告。

一、PEFT 到底是什么？用一句话说清楚

PEFT（Parameter-Efficient Fine-Tuning） = 不碰模型主体，只加"小补丁"来完成微调。

传统的微调（Fine-Tuning）需要更新模型的全部参数。GPT-3 175B 全量微调一次，等于是把 175B 个参数全重新训练一遍——需要数百块 GPU 跑几周，训练完还要存一份 350GB 的新模型。

PEFT 的思路完全不同：冻结原始模型不动，只在合适位置插入少量可训练的参数（通常不到 1%），训练时只更新这些"小补丁"。

	全量微调	PEFT
训练的参数	100%（175B）	0.1%–1%（≈100MB）
每任务存储	整个模型（350GB）	一个补丁文件（10MB）
推理延迟	不变	LoRA 变体：不变（可合并到原模型）
灾难性遗忘（学新任务忘旧任务）	有风险	**低**（原始模型冻结）

二、PEFT 三大路线

路线一：Adapter（适配器）

在 Transformer 每一层后面插入一个小神经网络模块（降维 → 激活 → 升维）。

```
Transformer 层
  ├── [Self-Attention]
  ├── [Adapter] ← 插入的小模块
  ├── [FFN]
  └── [Adapter] ← 插入的小模块
```

优点：可以在不同任务间切换（换 adapter 文件就行）。

缺点：推理时有额外计算（因为要串行通过 adapter）。

路线二：Prefix / Prompt Tuning（前缀/提示微调）

在模型输入或 hidden state 前面加一些"可学习的虚拟 token"——称为 **Soft Prompt（软提示）**。

```
输入：[prefix_1, prefix_2, ..., prefix_k, "翻译成英文：今天天气真好"]
                    ↑
              可学习的虚拟 token
```

这些虚拟 token 在训练时被优化，让模型学会"看到这个前缀就要做翻译"。

优点：极其轻量（0.01% 参数）。

缺点：在某些任务上效果不如 LoRA。

P-Tuning v2

Prompt Tuning 的一个改进版本（清华团队，2021.10）。核心改进：不只在输入层加 Soft Prompt，而是在每一层都加入可学习的 prompt token。这让 Prompt Tuning 在小模型和序列标注任务上也有效果。也是 HuggingFace 内置支持的方案。

路线三：LoRA（低秩适配）—— 事实标准

LoRA 的核心假设非常巧妙：**模型微调时权重的变化量（ΔW）是低秩的**。

什么意思？一个权重矩阵可能是 4096×4096 的，但每次微调产生的变化量可能只需要两个小矩阵就能表示——比如 4096×16 和 16×4096。

```
训练时：h = W₀x + BAx
          原始权重     低秩更新
                    ↑
               B 是 4096×16
               A 是 16×4096
               训练时只更新 B 和 A

推理时：把 BA 合并回 W₀
          h = (W₀ + BA)x
          速度完全不变！（这是 LoRA 胜出的关键）
```

LoRA 是 2026 年 PEFT 的事实标准，原因：

1. 合并后推理延迟零增加——Adapter 做不到这一点
2. 模型质量损失极小
3. 与量化兼容（QLoRA 把 LoRA + 4-bit 量化结合）
4. 训练参数减少 10,000 倍

三、量化（Quantization）—— 给模型"减肥"

量化是把模型的权重从高精度（16-bit）压缩到低精度（4-bit 或 8-bit）的技术。就像是把高清照片压缩成 JPEG——清晰度略有下降，但文件体积大幅缩小。

精度	每个权重占	70B 模型占用显存
FP16（标准）	2 字节	~140 GB
INT8（8-bit）	1 字节	~70 GB
NF4（4-bit）	**0.5 字节**	**~35 GB**

NF4 是 4-bit 量化的一种特殊格式（NormalFloat），专门为正态分布的权重设计，精度损失最小。量化之后的模型**存得更小、跑得更快**，但质量几乎不降。

四、QLoRA：让普通电脑也能微调大模型

QLoRA 是 LoRA 的升级版（Tim Dettmers, 2023），把三个技术组合在一起：

1. **NF4 4-bit 量化**：模型权重用 4-bit 存储，显存降到原来的 1/4
2. **双重量化（Double Quantization）**：连"量化的参数"本身也做一次量化，再省一点显存
3. **分页优化器（Paged Optimizers）**：显存溢出时自动把优化器状态暂存到 CPU 内存（类似电脑的虚拟内存）

结果：65B 模型用单块 48GB GPU（一块 A100）就能微调——这在 QLoRA 之前需要 8 块 GPU。

GPU 显存	QLoRA 能微调的最大模型
6 GB	7B（RTX 2060）
12 GB	13B（RTX 3080）
24 GB	**33B**（RTX 4090）
48 GB	**65B**（A100）

四、DoRA（权重分解低秩适配）—— LoRA 的升级版

DoRA（Weight-Decomposed Low-Rank Adaptation，NVIDIA, 2024）是 LoRA 目前最强的变体。它的洞察是：**权重的作用可以分解为"大小"和"方向"两部分。**

```
LoRA 同时改变方向和大小 → 粗放
DoRA：先把权重拆成"大小"和"方向"
      → 大小用可学习向量调
      → 方向用 LoRA 调
      → 更精细
```

效果持续优于 LoRA，在某些任务上接近全量微调。缺点是多了一个超参数（需要决定在哪几层做分解），使用比 LoRA 略复杂。HuggingFace PEFT 库已原生支持。

五、LoRA 的超参数

参数	推荐值	说明
r（秩）	8–64	越大效果越接近全量微调，但显存和参数也增加
alpha	2*r 左右	缩放系数
target_modules	"all-linear"	在哪些层上应用 LoRA

经验：r=16 是起点，大部分场景够用。需要更接近全量微调时用 r=64。

六、微调 vs RAG——什么时候怎么做

```
需要模型完成特定任务？
├─ 需要最新/外部知识 → RAG
├─ 需要格式/风格/行为改变 → 微调（LoRA/QLoRA）
├─ 需要提高领域准确度
│  ├─ 知识在文档中 → RAG
│  └─ 知识需内化（如医学诊断） → 微调
└─ 两者都需要 → 微调 + RAG（最常见最佳实践）
```

微调改变模型的"行为模式"和"知识内化"。RAG 提供"外部知识"。两者不是替代关系，是互补关系。

七、一张图总结

```
            PEFT 全景
                |
   ┌────────────┼────────────┐
   |            |            |
Adapter    Prefix/Prompt   LoRA（标准）
   |            |            |
插层小模块   虚拟token  低秩分解
   |            |            |
有额外延迟   极其轻量   零推理延迟
   |            |            |
   └────────────┼────────────┘
                |
            QLoRA（LoRA + 量化）
                |
     单卡微调 65B 模型

 2026年推荐路线：
 单卡内存够 → LoRA（r=16）
 单卡内存紧 → QLoRA（NF4 + LoRA）
 需要极致质量 → DoRA 或更高秩 LoRA
```